Anodite Energy Simulator

Technical Documentation

Real-time webcam hand tracking with Gwen Tennyson-inspired energy effects

Item	Detail
Language / runtime	Python 3.10 or 3.11 (MediaPipe does not support 3.14)
Libraries	OpenCV (capture, resize), MediaPipe Hands (landmarks), Pygame (window, drawing, UI), NumPy (math, particles)
Modules	main, camera, hand_tracking, gesture_recognition, energy_effects, renderer, ui, config
Platform	Windows (DirectShow camera backend, falls back to the default backend)
Privacy	Frames stay in memory, are processed locally and are never saved or uploaded

1. Architecture and data flow

The application is split into a capture/tracking thread and a render loop on the main thread. The worker publishes only its latest result, so a slow render frame never builds up a queue of old camera frames.

```
Webcam -> CameraWorker thread: read, mirror (cv2.flip), BGR->RGB,
          downscale to 480 px wide -> HandTracker (MediaPipe) -> smoothed Hand objects
                      |  latest (seq, rgb frame, hands)   [lock-protected]
                      v
Main loop: map landmarks to screen px (Renderer.to_screen) -> GestureEngine.update
          -> (HandState per hand, TwoHandState) -> EnergyEngine.update (spawn + simulate)
          -> Renderer: draw webcam, EnergyEngine.draw to additive layer, bloom, composite
          -> UI overlay -> display.flip
```

Module responsibilities

Module	Responsibility
main.py	App class: event loop, keyboard shortcuts, camera start/stop, gesture-event dispatch, fullscreen, dependency check, safe shutdown.
camera.py	CameraWorker: opens the webcam, runs capture + tracking on a daemon thread, exposes camera_ok / tracking_ok / error flags.
hand_tracking.py	HandTracker: MediaPipe Hands (or Tasks HandLandmarker fallback), per-hand exponential smoothing, stale-hand expiry.
gesture_recognition.py	GestureEngine: finger extension, palm orientation, pose smoothing, velocity/acceleration, thrust/sweep events, two-hand poses.
energy_effects.py	ParticleSystem, SpriteBank, trails, whips, projectiles, shockwaves, beams, two-hand fields, EnergyEngine orchestration.
renderer.py	Cover-fit webcam background, coordinate mapping, additive composite and bloom.
ui.py	Buttons, slider, quality and hand selectors, status dots, gesture guide, overlay text, error banner.
config.py	All colours, quality presets and gesture thresholds.

2. Camera and threading (camera.py)

- **Capture:** cv2.VideoCapture(index, CAP_DSHOW) requested at 1280x720, 30 FPS, buffer size 1 to keep latency low. If DirectShow fails the default backend is tried; if both fail, a helpful message about a missing

webcam or denied permission is stored in the error field.

- **Mirroring:** the frame is flipped horizontally once, before tracking, so MediaPipe's handedness label already matches the user's real hand and effects line up with the mirrored video.
- **Downscaling:** tracking runs on a 480 px wide copy (INTER_AREA); the full-resolution frame is still what is displayed.
- **Thread safety:** a Lock guards the shared (frame, hands, sequence number). The main loop reads it with latest(). A sequence number tells the main loop whether a new frame arrived, so gestures are only updated on new data.
- **Failure handling:** 60 consecutive failed reads set an error and stop the worker. A missing mediapipe package or tracker errors are reported in the UI instead of crashing. The finally block always releases the camera and closes MediaPipe.

3. Hand tracking (hand_tracking.py)

HandTracker wraps MediaPipe Hands (model_complexity 1, up to 2 hands, detection 0.6, tracking 0.5). If the installed MediaPipe build has no solutions module, it falls back to the Tasks HandLandmarker, downloading hand_landmarker.task once if it is missing. Each hand is returned as 21 landmarks (x, y normalised; z relative).

- **Label disambiguation:** with two hands detected they are sorted by x position and labelled Left/Right, because MediaPipe can occasionally report the same label twice. With one hand the MediaPipe label is used.
- **Smoothing:** exponential moving average per label: new = prev*(1-a) + raw*a with a = LANDMARK_SMOOTHING (0.55). The filter resets if the hand was gone for more than twice the grace period.
- **Graceful recovery:** a hand that stops being detected is kept for HAND_LOST_GRACE (0.25 s) and then removed, so brief tracking drops do not cause flicker or crashes.

4. Gesture recognition (gesture_recognition.py)

All distances and speeds are expressed in units of window height (H), so thresholds do not depend on window size. Landmarks arrive in screen pixels and are divided by H where needed.

4.1 Static features

- **Finger extension:** for each finger the ratio r = |tip - wrist| / |MCP - wrist| is computed. Score = clip((r - 1.25) / 0.55, 0, 1). An extended finger scores near 1, a curled finger near 0.
- **Pose scores:** OPEN_PALM = mean score weighted by the weakest finger; FIST = 1 - mean score, reduced if any finger is clearly extended; POINT = index score x (1 - mean of the other three).
- **Palm size:** average of wrist-to-middle-MCP length and 1.15 x knuckle-row width. This is rotation tolerant and is the hand's apparent size.
- **Palm centre:** mean of the wrist and the four finger MCPs. **Direction:** unit vector from wrist to middle MCP. **Point direction:** unit vector from index MCP to index tip.
- **Palm orientation (facing):** the 2D cross product of (index MCP - wrist) and (pinky MCP - wrist), divided by size squared and sign-flipped by handedness. Positive means the palm faces the camera, negative means the back of the hand. It is smoothed (EMA 0.4) with hysteresis (enter above 0.18, leave below 0.06). Hands showing their back are removed entirely, so no energy appears.

4.2 Temporal smoothing and hysteresis

Scores are smoothed with an EMA (GESTURE_SCORE_EMA 0.45). A pose is entered only when its score exceeds GESTURE_ENTER (0.62) for GESTURE_MIN_FRAMES (3) consecutive frames, and is kept while its score stays above GESTURE_KEEP (0.45). This prevents flicker between poses and false triggers.

4.3 Motion analysis

- Each hand keeps a deque of (time, palm position in H, ln(size)). Velocity is a finite difference over the oldest sample inside VELOCITY_WINDOW (0.12 s). Speed is smoothed 50/50 with its previous value; acceleration is the change in speed over time.
- **Forward thrust:** growth rate = d ln(size)/dt. A thrust fires when growth exceeds THRUST_GROWTH_RATE / thrust_scale (1.15 by default), lateral speed is below 1.6 H/s, the pose is not POINT and the 1.0 s cooldown has passed. A webcam has no real depth sensor, so apparent size growth is the proxy. Keys [and] change thrust_scale (0.4 to 2.5); Space fires a manual blast as a documented fallback.
- **Sideways sweep:** |vx| above SWEEP_SPEED (1.7 H/s), |vx| greater than 1.8 x |vy|, almost no size growth, 0.45 s cooldown. Emits sweep_left or sweep_right.
- **Fast movement:** speed between TRAIL_SPEED (0.9) and TRAIL_SPEED_MAX (3.5 H/s) is normalised to 0..1 and drives trail length and brightness.

4.4 Two-hand poses

Pose	Condition	Constant
BOTH_RAISED	Both wrists above 48% of window height and both hands open palm	RAISED_Y = 0.48
HANDS_TOGETHER	Palm distance below 0.55 H	TOGETHER_DIST = 0.55
HANDS_APART	Palm distance above 0.75 H and both hands have a pose	APART_DIST = 0.75

A candidate two-hand pose must persist for TWO_HAND_HOLD (0.12 s) before becoming active. The closing speed of the hands is tracked with an EMA. While HANDS_TOGETHER is active, the per-hand orbs fade out so only one shared orb is shown.

5. Gesture to power mapping

Gesture	Detected by	Power / effect
Open palm	OPEN_PALM pose, palm facing camera	Palm orb, swirling particles and arcs; charge grows over 4 s (CHARGE_FULL_TIME)
Close fist	FIST pose	Smaller, brighter orb, inward attractor, outward sparks, faster pulse; release burst when reopened after a long hold
Forward thrust	Size growth rate event	Projectile with bright core, particle trail, launch and impact shockwaves, impact burst
Sideways sweep	Lateral speed event	Whip: 26-point chain seeded from the palm's recent trail, flexible lag, fades over 1.1 s
Both hands raised	RAISED two-hand pose	Body aura, swirling particles, periodic shockwaves, arcs from hands to body
Hands apart	APART two-hand pose	Expanding field, rings, bowed strands and lightning between palms
Hands together	TOGETHER two-hand pose	One shared orb; size and brightness depend on distance and hold time
Index finger point	POINT pose	Jittering beam from the fingertip along the finger direction, sparks at the screen edge
Rapid movement	Speed above TRAIL_SPEED	Glowing wrist/palm/fingertip trails and fingertip sparks scaled by speed

6. Effects engine (energy_effects.py)

6.1 Particle system

ParticleSystem stores positions, velocities, life, max life and brightness in preallocated NumPy arrays and compacts dead particles with an index mask. Particle caps per quality preset: Low 1500, Medium 3500, High 7000, Ultra 12000.

- **Gravity-free motion:** every particle receives a sinusoidal turbulence acceleration based on its position and time, plus drag.
- **Attractors:** each hand and two-hand effect registers (x, y, pull, swirl, radius). Acceleration = direction x pull x falloff + perpendicular x swirl x falloff, where falloff = 1 / (1 + distance / radius). This gives orbiting, swirling attraction toward palms and orbs.
- **Colour over lifetime:** a 64-entry lookup table interpolates white-hot, light pink, pink, violet and deep violet from the particle's remaining life; alpha fades near the end.
- **Emission:** rates are multiplied by intensity x adaptive scale x a quality factor; fractional counts use random rounding so low rates still emit smoothly.

6.2 Procedural sprites and glow

SpriteBank generates three 256 px textures with NumPy: an orb (white core, pink body, violet halo), a soft halo and a swirl with spiral arms and noise. They are scaled with smoothscale, brightness-quantised into 8 levels and cached (cache cleared above 220 entries). Swirl sprites are rotated every frame for living motion. All sprites are drawn with additive blending (BLEND_RGB_ADD).

6.3 Lines, trails and lightning

- **Smooth curves:** trails use Catmull-Rom interpolation, vectorised with NumPy.
- **Glow polylines:** each line is split into tapered chunks and drawn in 4 layers (violet, magenta, pink, white). All queued lines are flushed layer by layer so bright cores are never overwritten by the dim halos of other lines.

- **Lightning:** recursive midpoint displacement with perpendicular random offsets, re-randomised every frame for a flickering look.
- **Ribbons:** wrist and palm trails get a travelling sine wave perpendicular to motion for a flowing, organic look.
- **Draw order:** lines, then circles, then additive sprites, so glows add on top of lines.

6.4 Orbs, projectiles, shockwaves and whips

- **Floating orb:** placed at palm centre + direction x palm size x ANCHOR_DISTANCE (0.75) with a small sinusoidal bob. Radii scale with palm size.
- **Projectile:** flies at 2.4 H/s in the movement direction (or finger direction for a toward-camera push). It emits trail particles each frame, shrinks if it is a depth push, and on leaving the screen or after 1.5 s triggers an impact burst plus two shockwaves.
- **Shockwave:** radius follows an ease-out curve, drawn as three concentric glow rings and a halo, optionally with an initial flash.
- **Whip:** a chain whose head tracks a nearby live hand for 0.45 s, then drifts; each segment follows its predecessor with an exponential lag that decreases along the chain.

6.5 Adaptive performance

Every 0.5 s the engine checks the smoothed FPS. Below ADAPTIVE_LOW_FPS (24) the emission scale is multiplied by 0.85 (minimum 0.25); above ADAPTIVE_HIGH_FPS (48) it recovers by 6% up to 1.0. The quality preset also sets the bloom downscale factor (8, 6, 5 or 4).

7. Compositing and bloom (renderer.py)

- **Background:** the camera frame is resized with a cover fit (scale = max(W/fw, H/fh)), centre-cropped, slightly darkened (alpha 0.78) and blitted as the base layer.
- **Coordinate mapping:** to_screen converts normalised landmarks using the same scale and offsets, so effects stay attached to the correct hand locations at any window size or aspect ratio.
- **Effect layer:** a black surface used as an additive 'transparent' layer. Sharp elements are drawn at full resolution.
- **Bloom:** the effect layer is downscaled, particles are accumulated into the low-resolution buffer with np.bincount (one call per colour channel), then the result is upscaled twice (normal and a wider, dimmer version) and added to the frame. Bilinear upscaling provides the soft glow cheaply.

8. User interface (ui.py)

- Custom widgets: Button, Slider (drag), quality and hand-mode selector buttons.
- Panel: START/STOP CAMERA, RESET EFFECTS, FULLSCREEN, GUIDE, intensity slider, quality selector, Left/Right/Both selector, camera and tracking status dots, FPS, particle count, adaptive scale, thrust sensitivity.
- The overlay shows the detected gesture and confidence for each hand, a fast-movement notice, the two-hand pose and a flash message for blasts and whips.
- Fullscreen cinematic mode hides the panel.

Key	Action
Enter	Start camera
F / F11	Fullscreen
ESC	Exit fullscreen, then quit
Q	Quit

Key	Action
H / G / R	Hide panel / guide / reset effects
Space	Manual blast
[and]	Thrust sensitivity down / up

9. Configuration reference (config.py)

Constant	Default	Meaning
CAMERA_WIDTH x HEIGHT / FPS	1280 x 720 / 30	Requested capture mode
TRACK_WIDTH	480	Width of the frame given to MediaPipe
DETECTION / TRACKING_CONFIDENCE	0.6 / 0.5	MediaPipe thresholds
LANDMARK_SMOOTHING	0.55	EMA weight of new landmark sample
HAND_LOST_GRACE	0.25 s	How long a vanished hand is kept
GESTURE_ENTER / KEEP / MIN_FRAMES	0.62 / 0.45 / 3	Pose hysteresis
FACING_ENTER / LEAVE	0.18 / 0.06	Palm-facing hysteresis
ANCHOR_DISTANCE	0.75	Orb offset beyond palm centre, in palm sizes
THRUST_GROWTH_RATE	1.15 /s	Size growth rate for a blast
THRUST_COOLDOWN / SWEEP_COOLDOWN	1.0 / 0.45 s	Anti-spam
SWEEP_SPEED	1.7 H/s	Whip trigger speed
TRAIL_SPEED / TRAIL_SPEED_MAX	0.9 / 3.5 H/s	Trail intensity range
RAISED_Y / APART_DIST / TOGETHER_DIST	0.48 / 0.75 / 0.55	Two-hand pose geometry
CHARGE_FULL_TIME	4.0 s	Hold time for full charge
QUALITY_PRESETS	1500 to 12000	Particle cap and bloom downscale

10. Error handling and resource cleanup

- A startup dependency check in main.py prints the exact pip command if a package is missing.
- Camera errors (no device, permission denied, stream lost, tracker failure) are surfaced as an on-screen banner instead of an exception.
- Hands leaving the frame are expired after a grace period; gesture and effect state for them is dropped and effects fade out.
- Shutdown: the main loop's finally block stops the worker thread (which releases the camera and closes MediaPipe) and calls pygame.quit().
- No frames are written to disk or sent over the network. The only optional network access is the one-time model download in the Tasks API fallback.

11. Known limitations

- Depth is estimated from apparent hand size, so forward thrust depends on lighting, distance and camera field of view and may need calibration.
- Palm-facing detection depends on MediaPipe's handedness label; a wrong label inverts the result.

- Thresholds were verified with synthetic landmark data and short live runs, and may need tuning per user and camera.